

ROYAUME DU MAROC

Ministere de l'Enseignement Superieur
Sciences de l'Ingenieur - Genie Informatique

PROJET DE FIN D'ETUDES

MyHeath

Plateforme de telemedecine et sante connectee

Backend ASP.NET Core 8 · Frontend React · MongoDB · Docker

Rapport technique et academique

Architecture, securite, realisation, Docker et deploiement

Annee universitaire 2025 - 2026

Document a destination du jury / encadrant

Resume / Abstract

Resume (FR)

MyHeath est une plateforme de telemedecine orientee sante feminine et masculine, avec suivi quotidien, diabete, cycle menstruel, messagerie chiffree, coach IA (Claude) et console d'administration (utilisateurs, abonnements). Ce rapport presente la version dont le backend est realise en ASP.NET Core 8 (C#), en conservant React (Vite) et MongoDB. L'accent est mis sur l'architecture en couches, la securite (JWT, BCrypt, AES-256-CBC), le deploiement Docker et les scenarios cloud (Vercel, Atlas, Azure).

Abstract (EN)

MyHeath is a telemedicine platform for women's and men's health tracking, diabetes care, encrypted messaging, AI coaching, and admin subscriptions. This report documents the ASP.NET Core 8 backend rewrite while keeping React and MongoDB, including Docker deployment and cloud hosting considerations.

Mots-cles

ASP.NET Core 8, React, MongoDB, Docker, JWT, AES-256, telemedecine, FemTech, Claude AI, Vercel, Azure

Table des matieres

1. Introduction et contexte	5
1.1 Problematique	5
1.2 Objectifs du projet	5
1.3 Pourquoi ASP.NET Core	5
2. Analyse des besoins	6
2.1 Acteurs	6
2.2 Exigences fonctionnelles	6
2.3 Exigences non fonctionnelles	6
3. Architecture technique	8
3.1 Vue d'ensemble	8
3.2 Stack technologique	8
3.3 Structure du projet .NET	9
3.4 Compatibilite avec le backend Node	9
4. Conception detaillee	10
4.1 Modele de donnees MongoDB	10
4.2 Securite AES-256-CBC	10
4.3 Analyse predictive	11
5. Realisation backend .NET	12
5.1 Authentification	12
5.2 Modules metier	12
5.3 Fonctionnalites ajoutees	13
6. Frontend React	16
7. Docker et deploiement	17
7.1 Conteneurisation	17
7.2 Variables d'environnement	17
7.3 Deploiement cloud	18
7.4 Pipeline recommande	18
8. Tests et comptes de demonstration	19

8.1 Scenarios de validation	19
9. Comparaison Node.js vs ASP.NET Core	20
10. Securite	21
10.1 Contre-mesures	21
10.2 Conformite	21
11. Conclusion et perspectives	22
11.1 Livrables	22
Annexe A - Exemples de requetes	23
Annexe B - Checklist Docker	24
Annexe C - Glossaire	25

1. Introduction et contexte

1.1 Problematique

L'accès à un suivi de santé continu (cycle, glycémie, bien-être) reste fragmenté entre applications isolées. MyHeath centralise le parcours patient-médecin-administrateur autour d'une API sécurisée et d'une interface moderne type dashboard.

1.2 Objectifs du projet

- Construire une API REST professionnelle en ASP.NET Core 8
- Conserver le frontend React existant (contrat /api compatible)
- Persister les données dans MongoDB
- Chiffrer les données sensibles (AES-256-CBC) et authentifier via JWT
- Ajouter RDV, rappels médicaments, notifications et audit admin
- Documenter Docker et le déploiement cloud

1.3 Pourquoi ASP.NET Core

Le choix d'ASP.NET Core s'aligne sur les compétences acquises en formation (C#, Web API, middleware, injection de dépendances, JWT). .NET 8 offre performances, typage fort, Swagger natif et un excellent support Docker (images officielles Microsoft).

2. Analyse des besoins

2.1 Acteurs

Acteur	Responsabilites principale
Patient	Suivi, diabete, IA, RDV, medic
Medecin	Consultations, RDV, messagerie
Administrateur	Users, plans, abonnements, aud

2.2 Exigences fonctionnelles

- Authentification (register/login) et RBAC patient | doctor | admin
- Suivi quotidien + score predictif 0-100
- Gestion des periodes / insights de cycle
- Dossier de sante chiffre et chat chiffre
- AI Coach (Anthropic Claude)
- Abonnements FREE / CARE / PREMIUM
- Rendez-vous telemedecine et rappels medicaments

2.3 Exigences non fonctionnelles

- Securite: JWT 7 jours, BCrypt cost 12, AES au repos
- Portabilite: Docker Compose
- Maintainabilite: Controllers / Services / Models
- Observabilite: healthcheck + Swagger

Figure 2.3 — Diagramme de cas d'utilisation

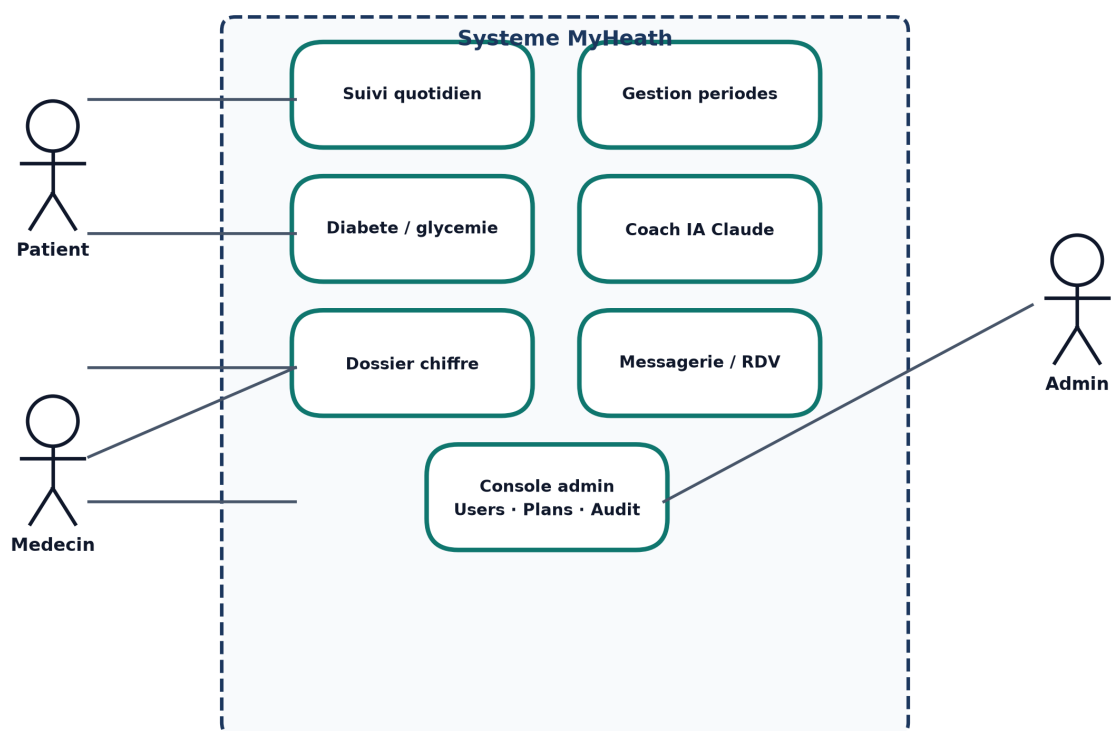


Figure 2.3 - Diagramme de cas d'utilisation

3. Architecture technique

3.1 Vue d'ensemble

Architecture 3-tiers: React (presentation) -> ASP.NET Core Web API (metier) -> MongoDB (persistance).
Le frontend appelle VITE_API_URL (exemple: http://localhost:5080/api).

Figure 2.1 — Architecture 3-tiers MyHeath (.NET + React + MongoDB)

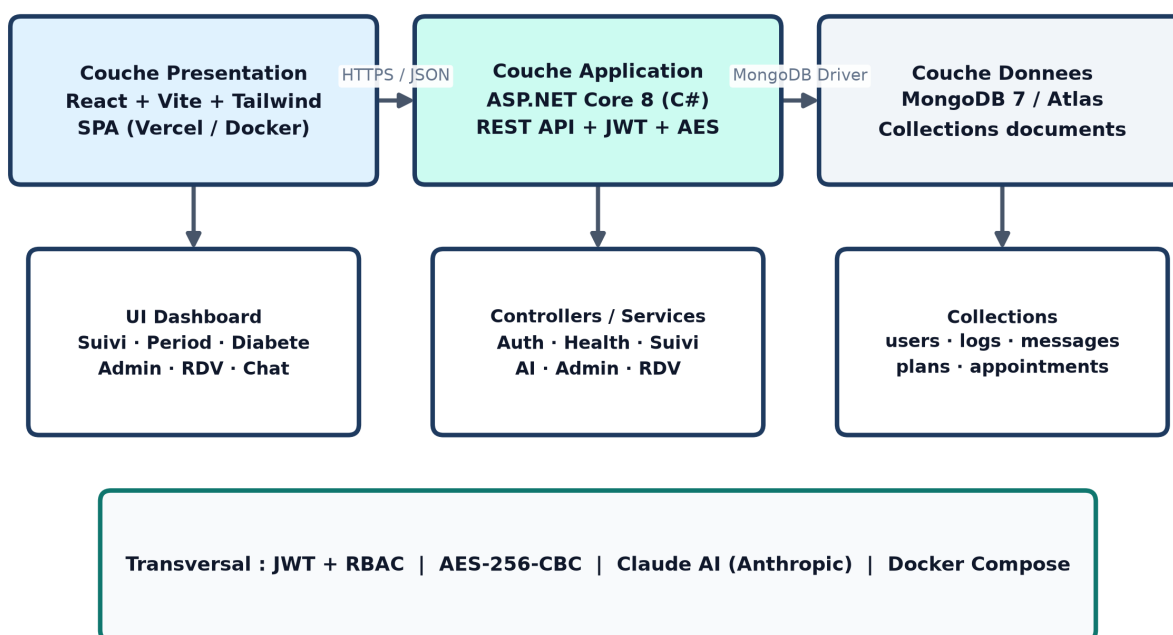


Figure 2.1 - Architecture 3-tiers MyHeath

3.2 Stack technologique

Couche	Technologies
Backend	ASP.NET Core 8, C#, MongoDB.Driver
Frontend	React 18, Vite, Tailwind, Rech
Donnees	MongoDB 7 (Docker) ou MongoDB
IA	Anthropic Messages API (Claude
Ops	Docker Compose, Vercel, Azure

3.3 Structure du projet .NET

```
backend-dotnet/  
  MyHeath.Api/  
    Controllers/  Auth, Health, Suivi, Chat, Ai, Admin, RDV...  
    Models/       Entities (User, DailyHealthLog, ...)  
    Services/     Mongo, AES, JWT, Claude, Seed, HealthScore  
    Program.cs    DI, CORS, Swagger, Authentication  
  Dockerfile  
docker-compose.dotnet.yml
```

3.4 Compatibilite avec le backend Node

Les routes REST restent sous /api/* avec des payloads JSON camelCase. Le JWT porte la claim id; AES utilise le format iv_hex:ciphertext_hex. Le meme frontend React peut basculer de Node (port 5000) vers .NET (port 5080).

4. Conception detaillee

4.1 Modele de donnees MongoDB

- users, healthrecords, symptomlogs, dailyhealthlogs, messages
- subscriptionplans, usersubscriptions
- appointments, appnotifications, medicationreminders, auditlogs

Figure 2.6 — Modele de domaines (extrait)

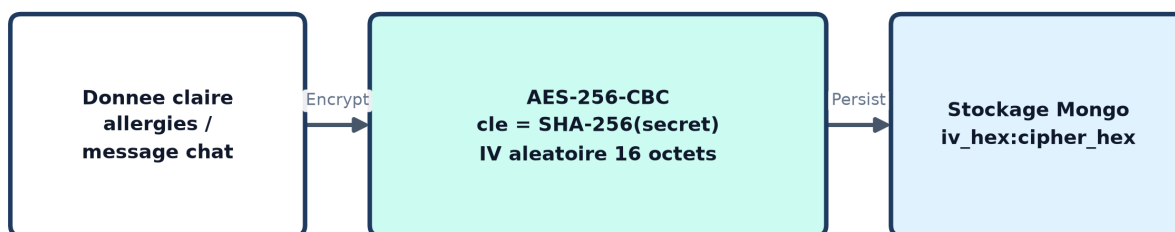


Figure 2.6 - Modele de domaines (extrait)

4.2 Securite AES-256-CBC

Les champs sensibles du dossier medical et le contenu des messages sont chiffres avec AES-256-CBC. La cle derive d'un secret via SHA-256, comme dans l'implementation Node, pour permettre l'interopabilite des documents existants.

Figure 2.7 — Chiffrement AES-256-CBC des donnees sensibles



Dechiffrement uniquement cote API apres JWT + RBAC

Figure 2.7 - Chiffrement AES-256-CBC

4.3 Analyse predictive

HealthScoreService calcule un score 0-100 a partir du sommeil, stress, activite, humeur et glycemie. CycleAnalyzer estime la phase, l'ovulation et les anomalies.

Figure 2.8 — Pipeline score sante & analyse de cycle

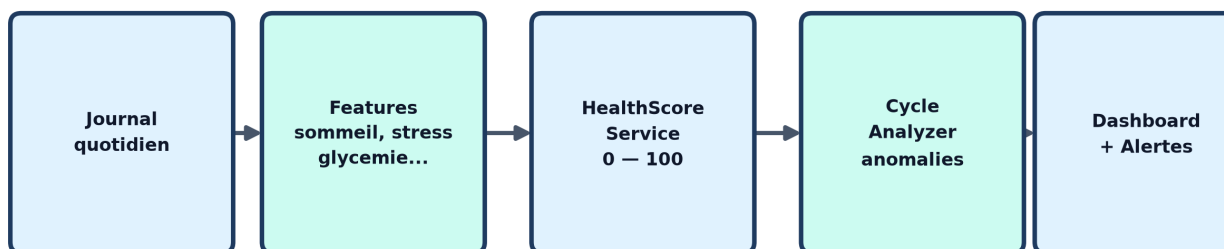


Figure 2.8 - Pipeline predictif

5. Realisation backend .NET

5.1 Authentication

AuthController expose register, login, me, doctors et assign-doctor. Les mots de passe sont hashés avec BCrypt (work factor 12). JwtTokenService emet un token HMAC-SHA256 valide 7 jours.

Figure 2.4 — Sequence : authentication JWT

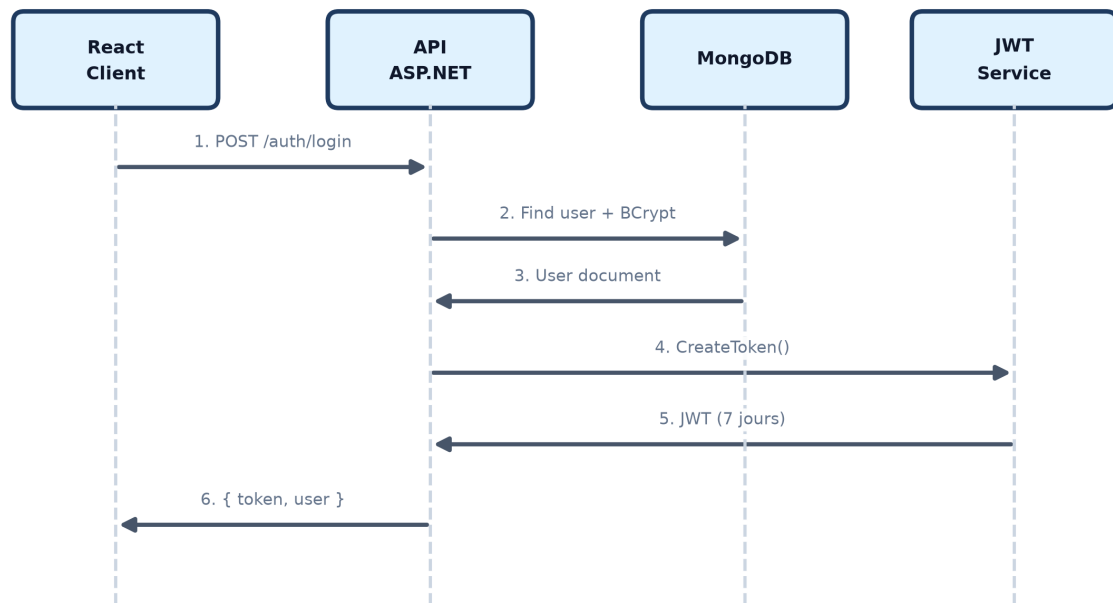
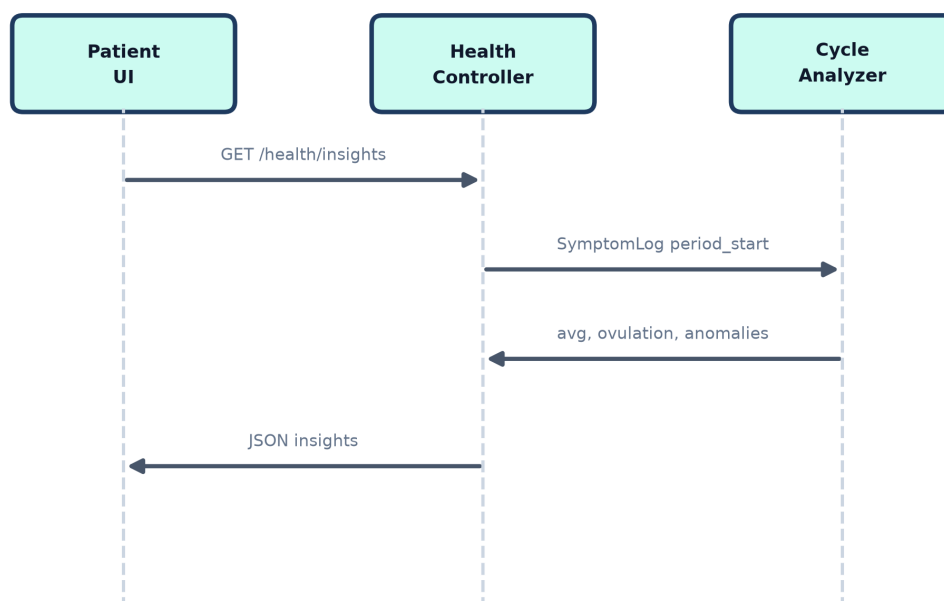


Figure 2.4 - Sequence d'authentification

5.2 Modules metier

- HealthController: symptomes, periodes, insights, dossier chiffre
- SuiviController: daily upsert, tendance, diabete
- ChatController: partenaires et messages chiffres
- AiController: chat, coach-plan, wellness, doctor-brief
- AdminController: users, plans, subscriptions, audit

Figure 2.5 — Sequence : calcul des insights cycle*Figure 2.5 - Sequence insights cycle*

5.3 Fonctionnalités ajoutées

Fonctionnalité	Endpoint
Rendez-vous	/api/appointments
Notifications	/api/notifications
Rappels médicaments	/api/medications
Audit admin	/api/admin/audit
Swagger / OpenAPI	/swagger

Figure 3.1 — Organisation du backend ASP.NET Core

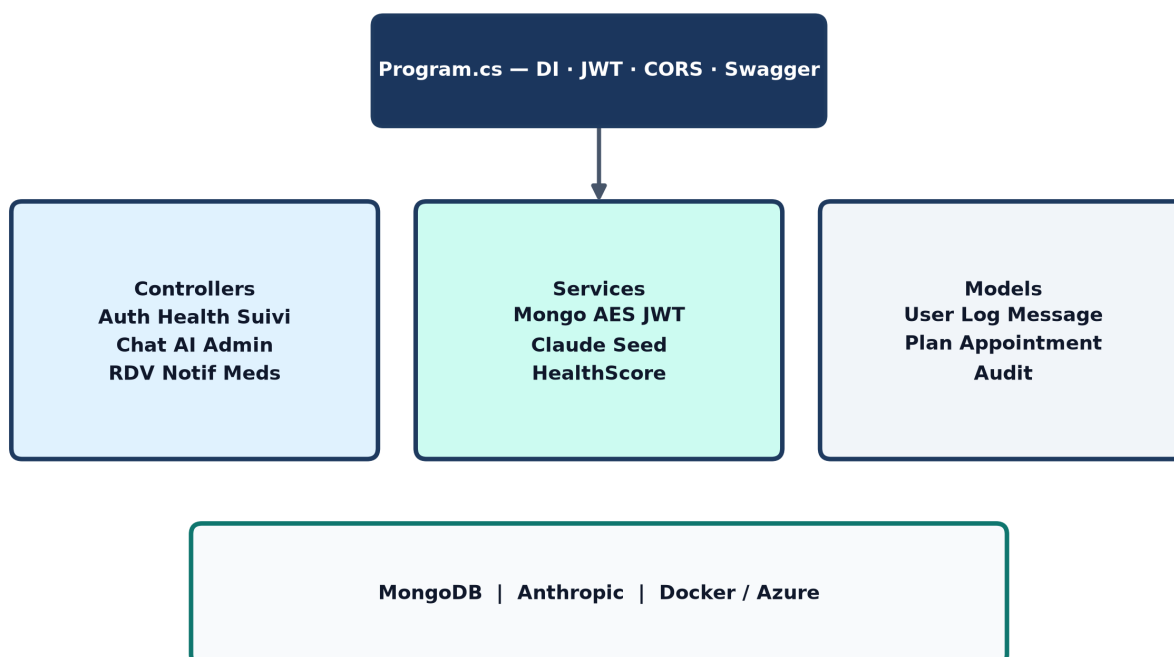


Figure 3.1 - Organisation backend ASP.NET Core

Figure 3.3 — Cartographie des routes API (/api)

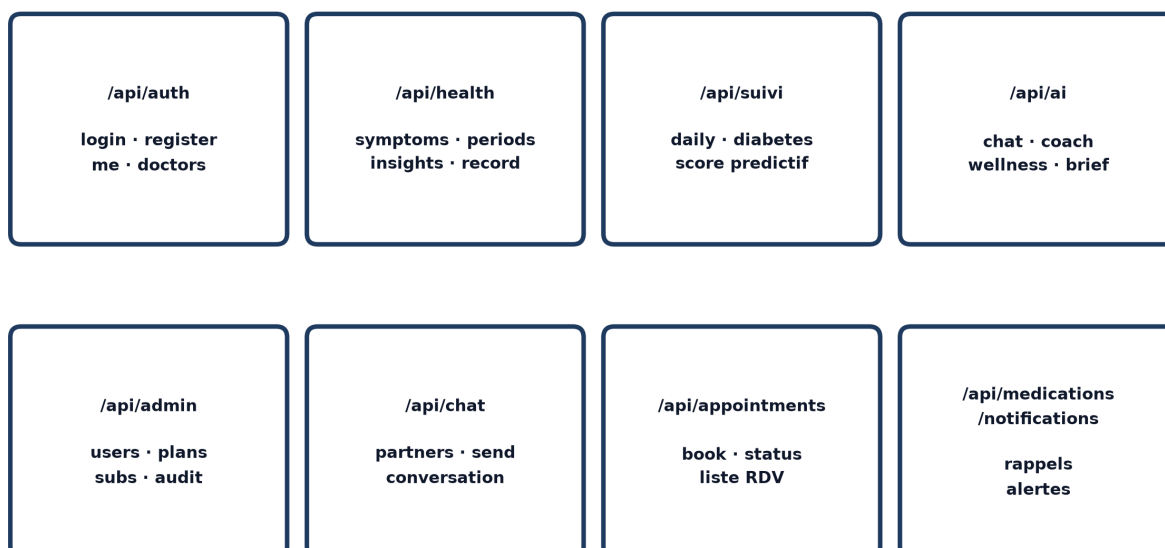
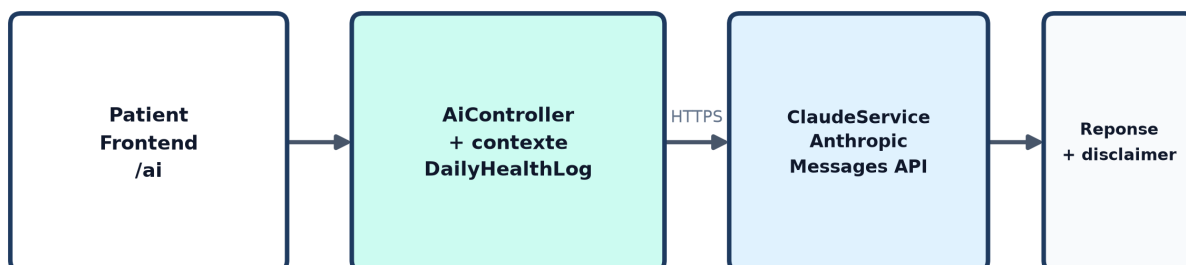


Figure 3.3 - Cartographie des routes API

Figure 2.9 — Integration MyHeath AI Coach (Claude)



Usage educatif uniquement — pas un diagnostic medical

Figure 2.9 - Integration AI Coach (Claude)

6. Frontend React

Le frontend conserve Vite, React et Tailwind avec un layout dashboard a sidebar. Les pages Appointments et Medications consomment les nouvelles routes .NET. La variable VITE_API_URL selectionne le backend (Node ou .NET).

- Admin: Overview, Users, Subscriptions, Plans
- Patient: Suivi, Period, Diabetes, Medications, AI Coach, Records
- Commun: Appointments, Consult (chat)

7. Docker et deployment

7.1 Conteneurisation

Le Dockerfile multi-stage utilise le SDK .NET 8 pour publier l'API, puis l'image runtime aspnet:8.0 pour exécuter MyHeath.Api.dll sur le port 5080. Le fichier docker-compose.dotnet.yml orchestre mongodb, backend-dotnet et frontend.

```
docker compose -f docker-compose.dotnet.yml up --build
# API      http://localhost:5080/api/healthcheck
# Swagger  http://localhost:5080/swagger
# UI       http://localhost:5173
```

Figure 2.2 — Topologie de deployment (Docker / Cloud)

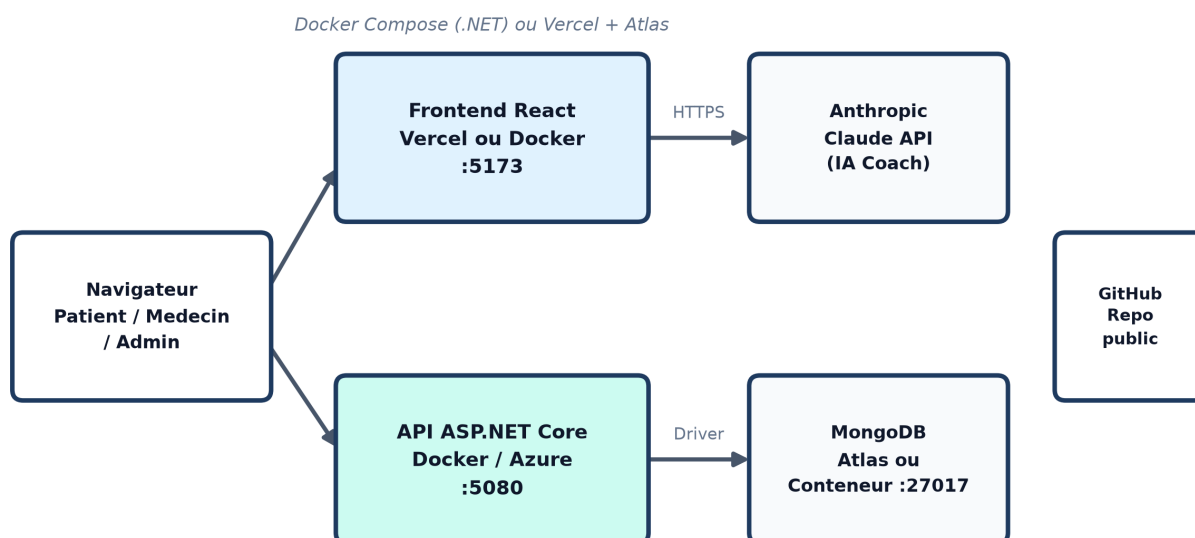


Figure 2.2 - Topologie de deployment

7.2 Variables d'environnement

- MONGODB_URI / MONGODB_DATABASE
- JWT_SECRET / AES_SECRET_KEY
- CLIENT_URL (CORS)
- ANTHROPIC_API_KEY / ANTHROPIC_MODEL (optionnel)
- PORT=5080

7.3 Deploiement cloud

Frontend: Vercel (build Vite, VITE_API_URL vers l'API publique). API .NET: Azure App Service, Railway, Render (Docker), ou VM avec Docker Compose. MongoDB: Atlas en production. Les secrets ne doivent jamais être committés dans Git.

7.4 Pipeline recommandé

- CI: build image Docker + smoke test /api/healthcheck
- CD: push registry puis deploy App Service / Compose
- Frontend: vercel --prod après mise à jour de VITE_API_URL

8. Tests et comptes de demonstration

Au démarrage, SeedService cree les comptes et plans s'ils sont absents.

Role	Email	Mot de passe
Admin	admin@myheath.app	Admin123
Medecin	doctor@myheath.app	Doctor123
Patient F	patient@myheath.app	Patient123
Patient H	man@myheath.app	Patient123

8.1 Scenarios de validation

- Login admin -> console -> assigner un abonnement
- Patient -> suivi quotidien -> score
- Patient -> book appointment -> notification medecin
- Patient -> medication reminder
- AI Coach (si ANTHROPIC_API_KEY presente)
- Chat chiffre patient-medecin

9. Comparaison Node.js vs ASP.NET Core

Les deux backends exposent le meme contrat API. Node (Express) reste utile pour un prototypage rapide et un deploiement serverless Vercel; .NET apporte typage fort, DI native, Swagger et une base solide pour des extensions entreprise.

Critere	Node.js	ASP.NET Core
Langage	JavaScript	C#
Typage	Dynamique	Fort / compile
Docs API	Manuelle	Swagger integre
Docker	node:alpine	mcr.microsoft.com/dotnet
PFE	MERN classique	Stack enseignee (.NET)

10. Securite

10.1 Contre-mesures

- HTTPS obligatoire en production
- Secrets via variables d'environnement (jamais dans Git)
- CORS restreint a CLIENT_URL + domaine frontend
- Validation des DTO et limitation de taille des prompts IA
- Desactivation douce des utilisateurs (isActive)

10.2 Conformite

Le chiffrement au repos des notes cliniques et messages s'aligne sur les bonnes pratiques Loi 09-08 (Maroc) et les principes RGPD. L'IA affiche un disclaimer non diagnostique.

11. Conclusion et perspectives

Ce PFE demontre la realisation d'une API telemedecine en ASP.NET Core 8 tout en preservant React et MongoDB, avec Docker et une strategie de deploiement claire. Perspectives: SignalR pour le chat temps reel, paiements d'abonnements, application mobile, et audits de conformite enrichis.

11.1 Livrables

- Code: backend-dotnet/, frontend/, docker-compose.dotnet.yml
- Rapport: docs/RAPPORT_PFE_MYHEATH_DOTNET.pdf
- Guide jury: GUIDE_INSTALLATION.md
- Depot public: <https://github.com/salahEddine-Admou/MyHeath>

Annexe A - Exemples de requetes

```
POST /api/auth/login
```

```
{ "email": "patient@myheath.app", "password": "Patient123" }
```

```
POST /api/suivi/daily
```

```
{ "sleepHours": 7.5, "energy": 7, "stress": 3, "mood": "good" }
```

```
POST /api/appointments
```

```
{ "doctorId": "...", "scheduledAt": "2026-08-01T10:00:00Z", "mode": "video" }
```

Annexe B - Checklist Docker

- Installer et demarrer Docker Desktop
- Cloner le depot GitHub MyHeath
- Executer: `docker compose -f docker-compose.dotnet.yml up --build`
- Verifier `http://localhost:5080/api/healthcheck`
- Ouvrir `http://localhost:5173` et se connecter
- En cas d'echec Mongo: verifier `MONGODB_URI=mongodb://mongodb:27017`

Annexe C - Glossaire

- JWT: JSON Web Token pour sessions API
- AES: Advanced Encryption Standard
- RBAC: Role-Based Access Control
- MRR: Monthly Recurring Revenue
- DI: Dependency Injection (.NET)
- Swagger: Documentation OpenAPI interactive
- Atlas: MongoDB Database-as-a-Service
- Compose: Orchestration multi-conteneurs Docker